Vector

The STL Vector is an automatically resizing dynamic array wrapper. When the vector runs out of space to hold new elements, it automatically resizes itself to be twice the size of its previous capacity. Items can be inserted into the beginning or middle of a vector while preserving order. To do so, all elements that come after the inserted element must be shifted. This happens automatically, as well.

Construction/Initialization

Vectors must have the type of data it is storing specified. An initial size can be provided to the constructor. Vectors support copying and assignment, like all STL containers. Like a C-array, a vector can be instantiated with a sequence of values in curly braces.

```
// need  to include vector
#include <vector>

int main() {
    // declaring an empty vector of integers
    std::vector<int> integers;
    // declaring vector with memory for 10 doubles allocated
    std::vector<double> doubles(10);
    // declaring vector of c-strings
    std::vector<const char*> months = {
        "Sunday". "Monday", "Tuesday", "Wednesday",
        "Thursday", "Friday", "Saturday"
    };
    // creating copy of months vector
    // C++17 added type deduction, so you don't need to use
    //  the template argument when it is obvious what the type should
    std::vector monthsCopy(months);
    // you can also use the equals sign to assign vectors
    //  but they must have the same type
    std::vector<int> otherIntegers;
    otherIntegers = integers;
```

```
    // can't do this, types mismatch
    // otherIntegers = doubles;
}
```

Adding Elements

The main way to add new elements to the end of a vector is via the `push_back` function. Using the subscript `[]` operator to add new elements is undefined behavior. It may work, it may not, depending on the compiler, and should therefore be avoided.

```
#include <iostream>
#include <vector>

int main() {
    std::vector<double> scores;
    double score = 0;

    while (score >= 0) {
        std::cout << "Enter score: ";
        std::cin >> score;
        scores.push_back(score);
    }

    return 0;
}
```

Accessing Elements

Vectors offer random access of its elements via the subscript operator `[]`. It is still possible to pass out of range elements to the subscript operator, just like a normal array. A member function `std::vector<T>::at(size_t index)` is used to check if the index is in range. If it is out of range, an exception is thrown. The size of the vector is accessed via the `size()` member function. The first element of the vector can be accessed via the `front()` member function, and the last element is accessed via the `back()` member function. If you need access to the underlying c-array, the `data()` member function can be

used. The `pop_back()` function can be used to remove the last element of the vector.

```
#include <iostream>
#include <vector>

int main() {
    std::vector<double> scores;
    double score = 0;

    while (score >= 0) {
        std::cout << "Enter score: ";
        std::cin >> score;
        scores.push_back(score);
    }

    std::cout << "There are " << scores.size() << " scores in the vect
    std::cout << "First score: " << scores.front() << "\n";
    std::cout << "Last score: " << scores.back() << "\n";

    for (int i = 0; i < scores.size(); ++i) {
        // accessing scores with the subsctipt operator
        std::cout << "Score " << i + 1 << ": " << scores[i] << "\n";
    }

    int lastIndex = scores.size() - 1;
    scores.pop_back();
    // won't cause an error, but data could be invalidated
    double lastScore = scores[lastIndex];
    // this will throw an exception
    lastScore = scores.at(lastIndex);

    return 0;
}
```

Before talking about the rest of the vector member functions, we need to discuss iterators.

Iterators

Before discussing the rest of the STL data types, we need to cover iterators. An iterator is a type of pointer that was developed to

standardize access to containers. An iterator is basically a pointer to an element of a container, however, there are additionally operations that can be performed on iterators to make traversing a container consistent no matter which type of container is being accessed. Below is a quick example of a vector being created, and an iterator being used to access its members:

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> values = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
    // the begin() member function returns an iterator to the first
    // element of a collection
    std::vector<int>::iterator it = values.begin();

    // we often use the "auto" type for iterators rather than writing
    // the end() member function returns the "past-the-end" iterator
    // of a collection. this iterator points to the location immediate
    // after the end of a collection
    auto end = values.end();

    // the end iterator is not referenceable, it is just a sentinel
    // value that marks the end of the container
    while (it != end) {
        // accessing the data an iterator is pointing to requires it
        // to be dereferenced using *, just like a pointer
        std::cout << *it << std::endl;
        // ++ is used to move the iterator to point at the next elemen
        ++it;
    }
    return 0;
}
```

Range-based for-Loops

One consequence of iterators is the ability to implement range-based for-loops. If you are familiar with for loops in Python or the `foreach` loop in C#, this concept should be familiar to you. The syntax for a range-based for-loop is below:

```
for (TypeName x : container) {

}
```

Below shows a traditional for-loop and a range-based for loop that performs the same operation:

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> values = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

    // tradition for loop
    for (int i = 0; i < values.size(); ++i) {
        std::cout << values[i] << std::endl;
    }

    // range-based for loop
    for (int x: values) {
        std::cout << x << std::endl;
    }

    return 0;
}
```

Vectors (continued)

Iterators can be used to insert at, or delete elements from, the middle of the vector. These operations are expensive, however, since they require shifting the entire vector to adjust for the missing or new element. The `insert()` function takes an iterator that points to the location the element will be inserted, and the element to be inserted. The element is inserted before the iterator the `insert()` function is passed. The function returns an iterator to the element that was just inserted. The `erase()` function is used to remove item(s) from a vector. It expects an iterator that points to the element to be removed. A start and end iterator can be used to erase a range of elements from

the vector. It returns an iterator to the next element after the last element that was removed.

Note: `insert()` and `erase()` invalidate any existing iterators to a vector. Since the items must be shifted, there is no guarantee that an iterator is pointing to the correct location in memory.

```
#include <vector>

int main() {
    std::vector<int> values = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    auto it = values.begin() + 2; // points to "3"
    values.insert(it, 2);
    // vector is now {1, 2, 2, 3, 4, 5, 6, 7, 8, 9, 10}
    // "it" is now invalid, so we need to reassign it
    it = values.begin() + 2;
    values.erase(it); // deletes the 2 that was just inserted
    // vector is now {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, x}
    // the last element is invalid, but still has memory allocated
    values.erase(values.begin(), values.begin() + 5); // erase first 5
    // vector is now {6, 7, 8, 9, 10}
}
```

Common Applications

`std::vector` is the standard container used in C++ for storing collections of data. If you aren't sure which of the other containers to use for an application, that typically means a vector will work fine. Vectors offer balanced performance for most applications.

Example

This example uses vectors to split a string and store the contents of a CSV file that contains information on orders:

```
#include <fstream>
#include <iostream>
#include <string>
#include <vector>
```

```cpp
// we expect the structure of the file to be
// John Doe,3,1000a,Filter

// struct to hold order information
struct Order {
    std::string customerName;
    int quantity;
    std::string itemId;
    std::string itemName;
};

// overloading << to print an order
std::ostream& operator<<(std::ostream& out, const Order& order) {
    out << "Customer:  " << order.customerName << "\n";
    out << "Quantity:  " << order.quantity << "\n";
    out << "Item ID:   " << order.itemId << "\n";
    out << "Item Name: " << order.itemName << "\n";
    return out;
}

// function to split a string on any character
std::vector<std::string> split(const std::string& line, char delim) {
    std::vector<std::string> parts;
    size_t start = 0;
    // find index delimitor in string
    size_t end = line.find(delim, start);
    // while end position of substring is not the past the end
    // of the string
    while (end != std::string::npos) {
        // push substring into vector
        parts.push_back(line.substr(start, end - start));
        // set the start of the next substring to the
        // next index of the string
        start = end + 1;
        // search for the next occurance of "delim"
        end = line.find(delim, start);
    }
    // push the last substring into the vector
    // if "delim" is never found, it will just
    // push the whole string into the vector
    parts.push_back(line.substr(start, end - start));
    return parts;
}

// main function
```

```
int main(int argc, char **argv) {
    // input filename needs to be provided via command line arg
    if (argc < 2) {
        std::cerr << "Must provide input filename\n";
        std::cerr << "Usage:\n";
        std::cerr << "\torder_report orders.csv\n";
        return 1;
    }

    // open and check the input file
    std::ifstream fin(argv[1]);
    if (fin.fail()) {
        std::cerr << "Could not open file: " << argv[1] << "\n";
        return 2;
    }

    // read the lines of the file into the split function
    // convert them to orders and add to the orders vector
    std::vector<Order> orders;
    for (std::string line; std::getline(fin, line);) {
        std::vector<std::string> orderParts = split(line, ',');
        // verify that the input is not malformed and we have the corr
        // number of parts
        if (orderParts.size() != 4) {
            continue;
        }
        orders.push_back({
            orderParts[0],
            std::stoi(orderParts[1]),
            orderParts[2],
            orderParts[3]
            });
    }

    // done with file stream -> close it
    fin.close();

    // print out the orders
    std::cout << "Orders:\n";
    // using range-based for loop to access elements
    for (const auto& order: orders) {
        std::cout << "------------------------\n";
        std::cout << order;
        std::cout << "------------------------\n";
    }
```

```
    return 0;
}
```

List

The STL List class is used to create a doubly linked list. Similar to a vector, lists store a collection of elements while preserving their order. Linked lists are different from vectors in that the elements can no longer be accessed by an index and are not stored contiguously in memory. This can be an advantage or disadvantage depending on the usage. Below is a table that compares lists and vectors.

Scenario	Vector	List
Insertion at End	May be instant, unless vector must be resized	Fast
Insertion at Middle	Requires shifting all elements that follow the insertion point, Slow	Fast
Insertion at Front	Requires shifting all elements, Slow	Fast
Deletion	Requires shifting of elements, Slow	Fast, and preserves iterators
Random Access (index)	Fast	Impossible

Scenario	Vector	List
Space	Does not need to be known at compile time, but resizing is expensive and can result in massive amounts of wasted space	Does not need to be known at compile time, and there is never wasted space

Similarities to Vector

Lists support many of the same operations as vectors, minus the subscript operator `[]`. Lists add the `push_front()` and `pop_front()`, in addition to the `push_back()` and `pop_back()` member functions. This allows direct access to the front of the list, whereas vectors only allow access to the back.

Unique Operations

Lists provide a few unique member functions for common list operations. First is `splice()`, which is used to combine two lists (or sub-lists) into one. Splicing allows elements of lists to be transferred between each other, or reordered within the same list, without the overhead of copying.

The `merge()` member functions assumes that you have two sorted lists, and merges them into a single sorted list.

The `remove()` function searches for the value that is passed to it, and removes it from the list. This is different from `erase()`, which expects an iterator to the element to be removed. The `remove_if()` function takes a function pointer/functor/lambda and removes items that satisfy the condition of that function. Don't worry too much about this for right now; function pointers/functors/lambdas are a point of discussion for next week.

Lists also offer a `reverse()` member function, that reverses the contents of the list, and `sort()` which sorts the list.

The `unique()` function removes consecutive duplicated elements. For

example, `std::list<int>({1, 2, 3, 3, 4, 5, 5, 2, 3}).unique()` would result in the list `{1, 2, 3, 4, 5, 2, 3}`. The consecutive 3s and 5s are removed, but the extra 2 and 3 at the end of the list stays.

Common Applications

Lists are used when there are frequent needs for insertions or removals, or to move elements via `splice()`.

Example

A cache is used to store recently accessed items. Many caches store use LRU (least recently used) to decide which item to evict from the cache when it gets full. One example of a cache is the CPU cache. Your computer's CPU has a small amount of memory that is faster than RAM that is part of the CPU. The most recently accessed data is stored in this cache, as it is assumed that recently used data will be accessed again soon. Another example of a cache is a browser cache. Recently accessed webpages, images, etc. are stored locally on your computer. This way, they will be available faster when you access them again.
This example shows a class that implements an LRU cache using a list as the internal cache representation. For the example we'll just use integers for data.

```
#include <iostream>
#include <list>

// class that implements LRU Cache using an std::list
class LRUCache {
public:
    LRUCache() {
        maxCacheSize = 64;
    }
    LRUCache(size_t size) {
        maxCacheSize = size;
    }
    // checks for a cache hit, i.e.,
```

```
    // if the data is in the cache
    bool hit(int data);
    // returns an iterator the data in the cache
    // if the data is not in the cache, it is added
    // if the data is in the cache, it is moved to
    // the front of the cache
    // if the cache is full, evicts LRU element
    // to make room for new item
    std::list<int>::iterator get(int data);
    // get the size of the cache
    size_t size() const { return maxCacheSize; }
private:
    // helper function to find an item in the list
    std::list<int>::iterator find(int data);
    // list for the cache
    std::list<int> cache;
    // limit to size of cache
    // if a cache is too big, it will become slower to
    // search and won't be as useful
    size_t maxCacheSize;
};

std::list<int>::iterator LRUCache::find(int data) {
    auto it = cache.begin();
    while (it != cache.end()) {
        if (*it == data) {
            return it;
        }
        ++it;
    }
    return it;
}

bool LRUCache::hit(int data) {
    return find(data) != cache.end();
}

std::list<int>::iterator LRUCache::get(int data) {
    // get iterator to data item
    auto it = find(data);
    // if it is found
    if (it != cache.end()) {
        // move to the front of the cache using splice
        cache.splice(cache.begin(), cache, it);
        // and return the iterator
```

```
        return it;
    }

    // if not, check if the cache is not full
    if (cache.size() < maxCacheSize) {
        // if it is not full, just push the new data to the front
        cache.push_front(data);
    }
    else {
        // remove the last item of the cache
        cache.pop_back();
        // add the data to the front
        cache.push_front(data);
    }

    // in both cases, we return the first iterator of the list
    return cache.begin();
}

int main() {
    LRUCache cache(4);

    cache.get(1);
    cache.get(2);
    cache.get(3);
    cache.get(4);
    cache.get(1);
    cache.get(5);

    if (cache.hit(1)) {
        std::cout << 1 << " - Cache hit\n";
    }
    else {
        std::cout << 1 << " - Cache miss\n";
    }
    if (cache.hit(2)) {
        std::cout << 2 << " - Cache hit\n";
    }
    else {
        std::cout << 2 << " - Cache miss\n";
    }
    if (cache.hit(3)) {
        std::cout << 3 << " - Cache hit\n";
    }
    else {
```

```
        std::cout << 3 << " - Cache miss\n";
    }
    if (cache.hit(4)) {
        std::cout << 4 << " - Cache hit\n";
    }
    else {
        std::cout << 4 << " - Cache miss\n";
    }
    if (cache.hit(5)) {
        std::cout << 5 << " - Cache hit\n";
    }
    else {
        std::cout << 5 << " - Cache miss\n";
    }
    return 0;
}
```

This will print:

```
1 - Cache hit
2 - Cache miss
3 - Cache hit
4 - Cache hit
5 - Cache hit
```
